

Automatic Grading and Feedback for Students' Short Written Responses

By

Aubrey Condor

A dissertation submitted in partial satisfaction of the
requirements for the degree of

Doctor of Philosophy

in

Education

in the

Graduate Division

of the

University of California, Berkeley

Committee in charge:

Professor Zachary Pardos, Chair
Professor Mark Wilson
Professor Marcia Linn

Spring 2024

PREVIEW

Abstract

Automatic Grading and Feedback for Students' Short Written Responses

by

Aubrey Condor

Doctor of Philosophy in Education

University of California, Berkeley

Professor Zachary Pardos, Chair

This work focuses on automatic grading and revision feedback for open-ended (OE) student responses relating to mathematics and science topics. The dissertation will be divided into three closely related chapters, where each build upon the work of the previous chapter(s). In the first chapter, we focus on improving the performance of an Automatic Short Answer Grading (ASAG) model, where performance is measured by how closely the machine ratings match those of a human rater. We experiment with incorporating domain-related text into the model with sources such as the item text, context, and rubric, and investigate how this might affect the generalizability of the model by testing the ASAG model on out-of-training-sample questions. Further, we examine if representing not only the text, but also the ordinal structure of a scoring rubric is useful by using a graph sampling method prior to incorporating the text into the ASAG model.

In the second chapter, we address the lack of explainability in typical ASAG models. We train a deep reinforcement learning (RL) agent to revise a student's response by inserting key phrases, or deleting portions of the response, to achieve a high grade from an ASAG model in the least number of revisions. The intent is that the agent's revisions may provide some explanation as to what was missing or unnecessarily included in a student's response. We also examine the intelligibility of a Neural Additive Model (NAM) using the RL agent's key phrases as input features. NAMs provide the explainability of additive models with the ability to approximate high dimensional functions and inputs like a neural network. Further, we use the RL agent to expose shortcomings in the ASAG model by finding revisions that achieve a high grade from the automatic grader but may not be considered good responses by a human rater.

In the third chapter, we focus on whether the RL agent's revisions can provide feedback to encourage a student to revise their own short response. We collected data with a randomized controlled design from students interacting with an online tutoring system. Students who receive an intervention from the RL agent's revisions will be compared with both students who receive an intervention from a popular open-source chatbot, ChatGPT, and students who received a generic intervention. We examine student perceptions of the feedback, how prior knowledge interacts with the interventions, and the accuracy of ChatGPT's feedback.

Table of Contents

List of Tables

List of Figures

Chapter 1

- 1.1 Introduction
- 1.2 Related Work
 - 1.2.1 Automatic Short Answer Grading
 - 1.2.2 Natural Language Processing for Education
- 1.3 Chapter 1, Study 1: Using Domain-related Text for Generalizability and Model Performance
 - 1.3.1 The Dataset
 - 1.3.2 Methods
 - 1.3.2.1 Input Representations
 - 1.3.2.1.1 Sentence-BERT
 - 1.3.2.1.2 Word2Vec
 - 1.3.2.1.3 Bag of Words
 - 1.3.2.2 Input Content
 - 1.3.2.2.1 Question Text
 - 1.3.2.2.2 Question Context
 - 1.3.2.2.3 Rubric Text
 - 1.3.2.3 Classification Models
 - 1.3.2.4 Evaluation and Model Comparison
 - 1.3.2.4.1 Experiments
 - 1.3.2.4.2 Leave-one-question-out Evaluation
 - 1.3.2.4.3 Evaluation Metrics
 - 1.3.2.4.4 Empirical Monte Carlo Confidence Intervals
 - 1.3.3 Results
 - 1.3.4 Discussion
 - 1.3.4.1 Next Steps
- 1.4 Chapter 1, Study 2: Utilizing a Scoring Rubric to Increase Model Performance
 - 1.4.1 Background
 - 1.4.1.1 BERT
 - 1.4.1.2 Node2Vec
 - 1.4.2 Methods
 - 1.4.2.1 The Dataset
 - 1.4.2.2 Architecture Modification
 - 1.4.2.4 Fine-tuning Experiments
 - 1.4.3 Results
 - 1.4.4 Discussion

Chapter 2

- 2.1 Introduction
- 2.2 Related Work
 - 2.2.1 Explainable AI and ASAG
 - 2.2.2 Reinforcement Learning for Education

2.2.3	Applications of Neural Additive Models
2.3	The Dataset
2.4	Chapter 2, Study 1: Neural Additive Models for Explainable ASAG
2.4.1	Background
2.4.1.1	Neural Additive Models
2.4.1.2	Logistic Regression
2.4.1.3	DeBERTa'
2.4.2	Methods
2.4.2.1	Feature Engineering
2.4.2.2	Evaluation
2.4.3	Results
2.4.4	Discussion
2.5	Chapter 2, Study 2: Training an RL Agent to Revise a Short Response
2.5.1	Background
2.5.1.1	Deep Reinforcement Learning
2.5.1.1.1	Offline (batch) Learning
2.5.1.1.2	Proximal Policy Optimization
2.5.1.2	The ASAG Model
2.5.2	The RL Formulation
2.5.2.1	The Algorithm
2.5.2.2	The State
2.5.2.3	The Action Space
2.5.2.4	The Policy
2.5.2.5	The Reward
2.6	Chapter 2, Study 3: Auditing an ASAG model with deep RL
2.6.1	The Experiments
2.6.2	Results
2.6.2.1	Experiment 1: Helpful Phrases
2.6.2.2	Experiment 2: Unhelpful Phrases
2.6.2.3	Experiment 3: All Phrases
2.6.3	Discussion
Chapter 3	
3.1	Abstract
3.2	Introduction
3.3	Related Work
3.3.1	Actionable Automated Feedback
3.3.2	AI-assisted, and Automated Feedback
3.3.3	Prior Knowledge and Feedback
3.3.4	ChatGPT for Education
3.4	Feedback Design
3.5	Methods
3.5.1	ChatGPT
3.5.2	The Data Set and Student Model
3.5.3	ChatGPT Prompt Design
3.6	Evaluation of the RL Agent and Machine Revisions
3.6.1	Quantitative Evaluation

- 3.6.1.1 Quantitative Evaluation Results
- 3.6.2 Qualitative Evaluation
 - 3.6.2.1 Qualitative Evaluation Results
- 3.7 Data Collection: Processing and Results
 - 3.7.1 Data Processing
 - 3.7.2 Summary Statistics
 - 3.7.3 Comparing Treatment Groups for Revision Efficacy
 - 3.7.3.1 Assumptions of ANCOVA
 - 3.7.3.2 ANCOVA Results
 - 3.7.3.3 Heterogeneity of Treatment Effect
 - 3.7.4 Examining Student Perceptions
 - 3.7.4.1 Engaging in Revision
 - 3.7.4.2 Similarity Analysis of Machine and Student Revisions
 - 3.7.4.3 Analysis of Student Critiques
 - 3.7.5 Analysis of ChatGPT Machine Revisions
- 3.8 Discussion and Limitations
- References
- Appendix: Screenshots of the Survey in OATutor

List of Tables

Table

- 1.1 Experiment Results: Multi-label Accuracy
- 1.2 Experimental Results: 95% Monte Carlo Confidence Intervals
- 1.3 Test Set Weighted F1 Score and Cohen's Kappa
- 1.4 5x2 Cross Validation Paired One-tailed t-test
- 2.1 KI Soundwave Scoring Rubric Sample
- 2.2 5x2 Cross Validation Paired One-tailed t-test and Cohen's D Effect Sizes
- 2.3 5x2 Cross Validation Averages
- 2.4 Pseudocode for the RL Algorithm
- 3.1 The IMR Construct Map
- 3.2 Scoring Guide for the OE Item
- 3.3 Summary Statistics for Number of Revisions
- 3.4 Summary Statistics for Construct Level Increase
- 3.5 Coded Likert Scale Averages and Standard Deviations
- 3.6 Counts of Participants Partitioned by Treatment and Demographic Groups
- 3.7 Summary Statistics of OE and Revision Scores by Group
- 3.8 ANCOVA Results
- 3.9 Results of Tukey's HSD Test
- 3.10 OLS Fit Results for the Baseline Model
- 3.11 Summary Statistics of Pretest Scores
- 3.12 OLS Fit Results for Model 2 with Prior Knowledge and Group Interaction

List of Figures

Figure

- 1.1 An Example of an Item from the Data Set
- 1.2 An Example of a Scoring Rubric Corresponding to the Item in Figure 1.1
- 1.3 2D Visuals of Input Vector Representations.
- 1.4 The Modified BERT Architecture
- 1.5 Example of a Scoring Rubric and Corresponding Graph Structure
- 2.1 KI Soundwave Item
- 2.2 NAM Mean Feature Importance
- 2.3 NAM Shape Functions (with highest and lowest rating categories)
- 2.4 NAM Shape Functions (with all rating categories)
- 2.5 A Comparison of the Smoothed Mean Episode Returns
- 2.6 Kernel Density Estimated Distribution Plots for each Experiment
- 3.1 The Four Building Blocks in the BEAR Assessment System
- 3.2 The Context and Graph for the Pretest Questions
- 3.3 The Context and Graph for the OE Item
- 3.4 Smoothed (n=200) mean Episode Returns
- 3.5 Frequency of Likert Agreement Levels for Statement 1
- 3.6 Frequency of Likert Agreement Levels for Statement 2
- 3.6 Frequency of Likert Agreement Levels for Statement 3
- 3.7 Histogram of ANCOVA Residual Values
- 3.8 Box Plot of Expected Revision Score by Treatment Group
- 3.9 Box Plot of Residuals by Treatment Group
- 3.10 Box Plot of Expected OE Score by Treatment Group
- 3.11 Scatterplot and Linear Fit for Expected OE versus Revision Score: RL Group
- 3.12 Scatterplot and Linear Fit for Expected OE versus Revision Score: Control Group
- 3.13 Scatterplot and Linear Fit for Expected OE versus Revision Score: GPT Group
- 3.14 Linear Fit of Covariate to Dependent Variable by Treatment Group

Chapter 1

Automatic Short Answer Grading: Using domain-related text for generalizability and model performance.

1.1 Introduction

Automatic Short Answer Grading (ASAG) is an emerging field of research, as the education community has started to embrace the use of machine learning to assist both students and education professionals. ASAG refers to the machine scoring of open-ended (OE) items that are shorter than essay length - often a sentence or two. It has been shown that the use of open-ended questions helps facilitate learning by self-explanation (Chi et al., 1994), or information recall (Bertsch et al., 2007). The “generation effect” has been studied extensively and findings show that subjects who generate information are able to remember the information better than material they have simply read (Bertsch et al., 2007). Additionally, open-ended items may be adapted to assess student knowledge and understandings across many content areas and have been shown to enhance learning (Hancock, 1995). However, educators are often deterred from the use of OE items because grading requires much more time than that for multiple choice items (Hancock, 1995). The use of an automated system for grading short answer questions could decrease the amount of time teachers spend grading, while allowing students to grow in their knowledge through self-explanation and information recall.

In this chapter, we focus on improving the performance of an Automatic Short Answer Grading (ASAG) model, where performance is measured by how closely the machine ratings match those of a human rater. We experiment with incorporating domain-related text into the model with sources such as the item text, context, and rubric, and investigate how this might affect the generalizability of the model by testing the ASAG model on out-of-training-sample questions. Further, we examine if representing not only the text, but also the ordinal structure of a scoring rubric is useful by using a graph sampling method prior to incorporating the text into the ASAG model.

1.2 Related Work

In this section, we give a brief background of research relating to ASAG and more broadly, applications of Natural Language Processing for educational purposes.

1.2.1 Automatic Short Answer Grading (ASAG)

A systematic review of trends in ASAG (Burrows, et al., 2015) illustrates an increasing interest in the field of automatic grading for education. Unsupervised methods have been explored such as concept mapping, semantic similarity, and clustering to assign ratings. For example, Mohler and Mihalcea (2009) compared knowledge based and corpus based semantic similarity measures for automatic grading, Klein et al. (2011) implemented a latent semantic analysis approach, and Basu et al. (2013) used clustering to provide rich feedback to groups of similar responses. In addition, many types of supervised classification methods have been utilized for ASAG. Notable examples include Hou and Tsao (2011) who incorporated POS tags and term frequency with a Support Vector Machine classifier, and Madnani et al. (2013) who made use of simple features such as a count of commonly used words and length of response with a logistic regression classifier.

More recent ASAG research exploits deep learning methods. Haller et al. (2022) provide a comprehensive analysis of recently published methods which deploy deep learning approaches for ASAG. They emphasize that the progression from engineered features of text to representation learning methods, including the use of word embeddings, sequential models and attention-based methods, has contributed to recent improvements in ASAG models. Notable work includes Zhang et al. (2016) who used a combination of feature engineering and deep belief networks, Liu et al. (2019) who employed multi-way attention networks, and Yang et al. (2018) who considered a deep autoencoder model specific to Chinese responses. Additionally, Qi et al. (2019) created a hierarchical word-sentence model with a CNN and Bi-LSTM model and Tan et al. (2020) explored the use of a graph convolutional network (GCN) to encode a graph of all student responses. Tulu et al. (Tulu et al., 2021) proposed a new approach to ASAG using a MaLSTM (a LSTM based method for creating sentence vectors).

Further, much of the newest ASAG work makes use of state-of-the-art transformer based models, including Gaddipati et al. (2020) who evaluated four different types of response embeddings, ELMo, GPT, BERT, and GPT-2 for their performance on an ASAG task, and Camus and Filighera (2020) who compared the performance of transformer models for ASAG in terms of the size of the transformer and the ability to generalize to other languages. Agarwal et al. (2022) introduced the Multi-Relational Graph Transformer (MitiGaTe) to prepare token representations that consider the structural context of a sentence and Zhu et al. (2022) propose a novel BERT-based, NN framework for ASAG by adjusting BERT embeddings with a semantic refinement layer consisting of a bidirectional-Long Short-Term Memory (LSTM) network and a Capsule network with position information.

More specifically related to our project, researchers have experimented with using extra, question-relevant information within ASAG models. Sung et al. (2019; 2019) examined the effectiveness of pre-training BERT on relevant domain texts, as a function of the size of training data and generalizability across domains, and found that the pre-trained BERT model can be improved by augmenting data on domain-specific texts. Wang et al. (2021) used an attention-based method to extract key phrases from student answers that are highly related to phrases and answers from a scoring rubric. They find that the performance of both grading and justification identification is improved through their method. Chen and Li (2021) make use of prior knowledge to enrich features of student responses by incorporating question text information within an extra forward propagation training step. They show that by incorporating this question-specific information, their model beats current state-of-the-art performance.

1.2.2 Natural Language Processing for Education

Literature addressing the general application of natural language processing (NLP) for uses in the field of education has grown in recent years. Shaik et al. (2022) provide a review of the trends and challenges of using existing NLP methodologies for educational domain applications like sentiment annotations, entity annotations, text summarization and topic modeling, and Ahadi et al. (2022) analyze the metadata of publications using text mining or NLP in education settings and report on key themes as well as current state-of-the-art methods. More specific applications include Fonseca et al. (2020) who used NLP to automatically classify the programming assignments for students within academic context, and Thaker et al. (2020) who incorporated textual similarity techniques to recommend remedial readings to students. Further, Arthurs and Alvero (2020) examined bias in word representations for college admissions essays, and Xiao et al. (2020) employed NLP and transfer learning methods for problem detection in

peer assessments. Finally, Dowell and Kovanovic (2022) describe the current landscape of NLP tools and approaches to discern patterns in conversations that take place across educational technology platforms.

1.3 Chapter 1, Study 1: Using Domain-related Text for Generalizability and Model Performance

This section contributes to the field of automatic grading in three related ways. We focus on classification performance and generalizability of the supervised grading model in terms of 1) the textual representation type, 2) the content of the input and 3) the classification model.

A key challenge with supervised automatic grading models, where supervised learning refers to a sub-category of machine learning defined by the necessity of labeled data sets to train a model, is gathering a large enough sample of labeled data for training. While some labeling tasks for supervised learning may be straightforward such as identifying an image as either a dog or a cat, others like rating student responses require careful consideration. In high-stakes assessment scenarios, two or more ratings by different experts are often necessary to form a reliable consensus rating. Thus, obtaining labeled data to train an ASAG system can be arduous. It follows that quickly introducing new questions to an existing system may not be feasible if a data collection as well as the meticulous rating of new responses is necessary. Further, a new model would have to be trained and tuned with the newly collected responses. If we create more generalizable ASAG models, educators may have the flexibility to add new questions to an existing assessment with very little effort, thus increasing the practical use of the ASAG system.

We hypothesize that the inclusion of extra question related information within the model input may improve both the classification performance, and the generalizability of the model. For the purposes of this project, we formally define the generalizability of an ASAG model as the capacity to classify responses from out-of-training-sample questions, and the classification performance represents alignment with human scores.

We compare three different representation types, including those of state-of-the-art models: Sentence-BERT (SBERT), Word2Vec (W2V), and Bag of Words (BOW). In terms of input content, we experiment with including previously untapped resources relating to the questions in the model input. Such resources include a question-bundle identifier, the question stem text, question context text, and rubric information. Extra input content is vectorized (if the source is textual) and concatenated to the response vectors to be used as input to the classification model. Finally, we compare a non-neural model (a multinomial logistic regression) and a simple neural model (a three layer feed-forward network).

In order to examine the generalizability of the model for each experiment, we use a leave-one-question-out evaluation procedure where we train the model on $N-1$ questions and use the one left-out question data as our test set. We repeat this method $N=31$ times for each question, and reported accuracies are the average of the N experiments. Additionally, we compare results of our experiments against a typical random, student level $N=31$ -fold cross validation. We randomly partition the data into 31 splits, and use one as the test set, and the rest of the data is used for training. This is repeated $N=31$ times such that each N th partition is used as the test set once, and reported results are the average of the N holdout sets. Results from a majority class classifier are included as well for a baseline comparison.

Finally, we use each of the neural network leave-one-question-out experimental results (accuracies) as an observed distribution on which to employ Monte Carlo sampling and compute empirical confidence intervals of the sampling means. To compare results over the different

vectorization methods and different variations of included question-related information, we investigate if the computed empirical confidence intervals of the sampling mean accuracies overlap between the different experiments. As detailed in the results, highly overlapping confidence intervals suggest that we do not have strong evidence to support the hypothesis that adding extra question-related information helps the auto-grader achieve higher accuracies when grading out-of-sample questions.

The novel contribution of this section includes both the focus on generalizability of the model to out-of-training-sample questions, as well as the leveraging of previously untapped, question related information as input to the model.

1.3.1 Data Set

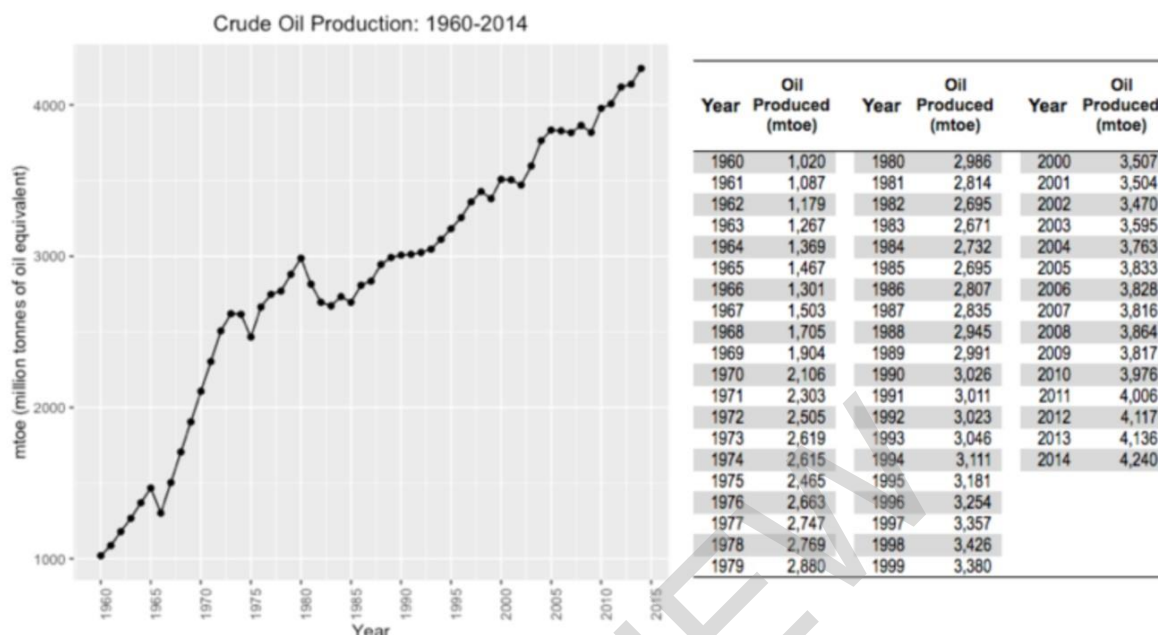
The data we used for this project was sourced from a 2019 field test of a Critical Reasoning for College Readiness (CR4CR) assessment (Arneson et al., 2019) created at the Berkeley Evaluation and Assessment Research (BEAR) center. The data consists of 5,550 student responses from 558 distinct students to 31 different items. The field test included other items that were multiple choice, but these questions were filtered out of the data for our use. The mean number of responses per question is 179 with the minimum being 128 and the maximum being 313. Most of the items belong to an item bundle - a grouping of items that are related in context and/or share a common question context. Additionally, the items were administered in four different test forms, where some items were included in multiple forms. The items all relate to four constructs about student understanding of algebra.

An example of one of the items, labeled 'Crude Oil 4ab' is included in Figure 1.1. For this item, students are presented with two images relating to oil production - one being a line graph and the other, a table. With the given context, students are presented with a choice between the graph or the table for which would be better to represent the historical patterns, or change over time of the oil production. The correct answer for this question is the graph, and students are expected to provide reasons why this is the right choice.

Figure 1.1

An Example of an Item from the Data Set: Crude Oil 4ab

The following graph and table show annual crude oil production in million tonnes of oil equivalent (mtoe) from 1960 to 2014.



[a] For a group project, you and your classmates have to present the overall historical patterns of annual oil production as a poster. Due to limited space, only one of the following representations can be included in the poster: a table, a graph, or a set of equations. Which representation should you use?

[b] Briefly explain your answer choice in [a].

An example of a student response to the Crude Oil 4ab question (shown in Figure 1) rated at the highest (most correct) score category is shown:

"The graph easily displays patterns over time whereas the table and equations require more analyzing."

In contrast, a student's response to the same question rated at the lowest (most incorrect) category is shown:

"The table is more clear, the information is seen in the table."

Responses were rated from 0 (fully incorrect) to 4 (fully correct) by multiple researchers and subject-matter experts at the BEAR center. The quality and consistency of ratings were evaluated by an inter-rater reliability score, and when a high percentage of rating mismatches between raters existed, incongruous ratings were discussed until a consensus was reached by the raters.

1.3.2 Methods

In this section, we briefly introduce the representation methods and model classes that we include in our experiments. Additionally, we describe the question related information, beyond that of the responses, that are used as inputs to the model. Further, we outline our experiments in detail including methods for evaluation and comparison.

1.3.2.1 Input Representations

We chose to include three distinct, yet commonly used in the field of natural language processing (NLP), representation types in our experiments: A count-based method that elicits the distinct vocabulary of our data (Bag of Words), a simple neural method that utilizes pre-trained word vectors (Word2Vec), and a state-of-the-art, contextual neural method (Sentence-BERT). A short description of each representation type is included.

1.3.2.1.1 Sentence-BERT

Sentence-BERT (SBERT) is a modification of the BERT network that utilizes siamese and triplet network structures to create semantically meaningful sentence embeddings (Reimers & Gurevych, 2019). SBERT re-tunes the BERT network on a combination of the SNLI dataset and the Multi-Genre NLI datasets, totaling about one million sentence pairs. Although sentence embeddings can be derived from the original BERT model using methods such as averaging the BERT output layers or using the “[CLS]” token embedding, it has been shown that such methods yield poor sentence embeddings (Pennington et al., 2014). In comparison, SBERT sentence embeddings outperformed other state-of-the-art methods such as InferSent (Conneau et al., 2017) and Universal Sentence Encoder (Cer et al., 2018) on the SentEval (Conneau et al., 2017) benchmark, which gives an idea of the quality of sentence embeddings for various task.

1.3.2.1.2 Word2Vec

Word2Vec (W2V) is a neural model that creates vector representations of words that have been shown to be semantically meaningful and useful in different NLP tasks (Mikolov et al., 2013). We use an extension of the previously introduced Skip-gram model (Mikolov et al., 2013) that incorporates sub-sampling of frequent words during training in order to speed up training, and improves accuracy of representations of less frequent words. For this project, we use the Google News corpus of pre-trained word embeddings. Vectors of size 300 are created for each word, and in order to construct response embeddings from the individual word vectors, we employ a simple yet popular method: averaging the vectors of all words in the response.

1.3.2.1.3 Bag of Words

The bag-of-words (BOW) model (Harris, 1954) represents a document as a vector, or “bag,” of length equal to the number of unique words in the entire corpus and values of the vector equal to the frequency with which its corresponding word occurred in the document. In our application, the bags are student short answers and additional question information.

1.3.2.2 Input Content

In an item design context, there are various, typically untapped sources of information relating to a particular question that may be useful to include as input to a classification model. We explore the use of four different sources of information, outside that of the response itself. A brief description of such sources are included below.

1.3.2.2.1 Question Text

The question text consists of the direct question stem. As in the example item provided in Figure 1.1, the question text would be: *“Briefly explain your answer choice in [a].”*

1.3.2.2.2 Question Context

The question context includes any textual information beyond that of the question stem that is related to the question, and might be useful for the respondent to produce a response. In the question example in Figure 2, the question context would include:

"The following graph and table show annual crude oil production in million tonnes of oil equivalent (mtoe) from 1960 to 2014. For a group project, you and your classmates have to present the overall historical patterns of annual oil production as a poster. Due to limited space, only one of the following representations can be included in the poster: a table, a graph, or a set of equations. Which representation should you use?"

We note that not all of the included items have question context text beyond that of the question text itself. For such items, it was not possible to include question context as part of the input.

1.3.2.2.3 Rubric Text

An important step in a measurement process is defining the outcome space for each item through a scoring guide, which is essentially a rubric. The scoring guide includes a detailed description of the reasons for which a response would be rated at a certain score and is used as a guide for human raters. In addition, the scoring guide often includes example responses for each rating level. An example of a scoring guide for the 'Crude Oil 4ab' item in Figure 1.1 is included shown in Figure 1.2. When the rubric text is included in the input text, we include both the level descriptions as well as the example response(s).

Figure 1.2

An Example of a Scoring Rubric Corresponding to the Item in Figure 1.1

Level	Description	Example Response
4	Student provides a fully correct positive and negative justification for ...	"The graph best illustrates the visual trend of oil production. The table or the equation won't provide an overall ..."
3	Student provides a fully correct positive justification for selected representation ...	"The table cannot show a trend in the oil production effectively. "
2	Student provides a partial/general positive justification for selected ...	"The graph helped me with more questions."
1	Student provides incorrect justification for selected representation.	"Because the graph gives us better projections."
0	Student makes no attempt to provide a justification for selected representation.	"I am not sure why."

1.3.2.2.4 Bundle Identifier

As described above in the data section, most of the questions belong to a bundle of questions - those that are linked based on a similar image, or context text. As a question bundle identifier, we concatenate a one-hot vector to the input vectors. Although items within the same bundle will often share the same context text, we include the one-hot bundle identifier

within our extra input text experiments so that we can infer whether the model makes use of semantics within the context text, or rather just a general indication of similar questions.

1.3.2.3 Classification Models

We compare a multinomial logistic regression model with a simple neural network classification model. We chose these classification methods representing a linear transformation of the feature space to a label (regression) and a non-linear transformation (neural network). A brief overview of each model is included.

Multinomial logistic regression (MLR) is a classification model that predicts probabilities of different outcomes for a categorical dependent variable. In order to generalize to a K-class setting, the model runs K-1 independent binary logistic regression models where one outcome is chosen as a “pivot” and other K-1 outcomes are separately regressed against the pivot outcome. We use the Limited-memory Broyden-Fletcher-Goldfarb-Shannon (LBFGS) algorithm for optimization (Fletcher, 2000), and incorporate L2 regularization.

Additionally, we use a simple feed forward neural network on a categorical cross entropy loss function with 2 hidden layers of size 100, using rectified linear unit (ReLU) activation functions for both hidden layers. We include a dropout of 0.4 and utilize Adam optimization. We train the model for 16 epochs and use a batch size of 36.

1.3.2.4 Evaluation and Model Comparison

In this section, we enumerate our experiments and the evaluation methods chosen for comparison.

1.3.2.4.1 Experiments

As input to our model, we experiment with eight different combinations of content to vectorize and concatenate to the response vectors before training our classification models:

- 1) response
- 2) question + response
- 3) question context + response
- 4) scoring rubric + response
- 5) bundle one-hot + response
- 6) question + scoring rubric + response
- 7) bundle one-hot + question + scoring rubric + response
- 8) bundle one-hot + question + question context + scoring rubric + response

For each of the 8 combinations listed above, we create three different vector representations with the aforementioned methods: 1) Sentence-BERT (SBERT), 2) Word2Vec (W2V), and 3) Bag-of-words (BOW), resulting in 24 distinct input types. We fit a classification model for each of the input types, for both of our classification models. Thus, we compare 48 separate versions of an ASAG model with three types of evaluation.

1.3.2.4.2 Leave-one-question-out Evaluation

In order to assess the generalizability of the ASAG model to out-of-training-sample questions for each of the 48 experiments, we average the results of N=31 (where N is the number of questions) independent models. For each of the N models, we train the classifier on data from N-1 questions, and test on data exclusive to the left-out-of-training question. In the case of the leave-one-question-out results, it is important to note that although the model has not seen

data specific to the left-out question, it has seen questions that are part of the same question bundle and are therefore related.

For comparison, we also include results from a standard N -fold cross validation. We chose $N=31$ (number of questions) so that the test set sizes were comparable to those in the leave-one-question-out experiments, although not exact since each question has a unique number of responses. Our data is randomly partitioned into N splits, and for each $1/N$ sized partition, we use the one partition as the test set, and the remaining $N-1$ partitions as the training set. This is repeated N times such that each partition is used as a test set once, and reported results are the average of the N experiments. This allows us to evaluate the model in such a way that is not specific to one particular training data set.

1.3.2.4.3 Evaluation Metrics

We report our results in multilabel accuracy because it is both widely used and easy to interpret. Multilabel accuracy represents the degree to which our model classifications agree with the ground truth labels (for this project, human ratings). It is calculated simply as the number of correct predictions divided by the total number of examples.

1.3.2.4.4 Empirical Monte Carlo Confidence Intervals

We conceptualize the results of each leave-one-question-out experiment as data points in a distribution - for each experiment we have a distribution of 31 data points. Thus, we are able to simulate Monte Carlo sampling distributions from each observed distribution in order to construct a sampling distribution of sample means. We specifically focus on the neural network models to create the Monte Carlo Confidence Intervals in order to have a more robust comparison between models. For simplicity, we do not calculate the confidence intervals for the logistic regression experiments because it appears almost universal across results that the neural network models perform better.

For each leave-one-question-out experiment, we calculate a cumulative distribution function from the empirical probability density function by ordering the unique observations in the data sample and calculating the cumulative probability for each as the number of observations less than or equal to a given observation divided by the total number of observations. We then sample from this distribution by drawing a random number within the range of our observed distribution, and getting the minimum CDF density where it is greater than the random draw. We sample $M=1000$ distributions of size $N=31$ (number of questions). For each distribution, we calculate the mean value. We thus have a distribution of sample means that is size 1000 for each neural network experimental setup.

Empirical 95% confidence intervals are calculated with the lower bound as the 2.5th percentile of the 1000 means, and the upper bound as the 97.5th percentile of the 1000 means for each neural network experimental model. So, we know from direct observation that 95% of our sampled means fall within the CI reported. In order to justify the conclusion that one model is superior to others, the lower bound of the empirical confidence interval should be higher than the upper bound of the CI for the model to which we are comparing.

1.3.3 Results

Results of our experiments are detailed in Table 1.1, reported in multi-label accuracy. In the left-most part of the table, an “X” is present for a given row if the information type, indicated by the column header, is included in the model input. For example, results in the first row

represent an input of only the response text and results in the second row represent an input of both the Bundle ID and the response. Additionally, the top half of the table results are those from the multinomial logistic regression classifier, and the bottom half of the table results are those for the neural network classifier (as indicated by the leftmost column). For each of our evaluation methods, random holdout, question holdout, and bundle holdout, we present results for the three textual representation methods: SBERT, W2V, and BOW. Finally, as previously mentioned, for the neural network experiments in the bottom half of the table, we present 95% empirical confidence intervals from Monte Carlo sampling.

Table 1.1
Experiment Results: Multilabel Accuracy

	Resp Text	Bundle ID	Quest Text	Context Text	Rubric Text	Random Holdout			Question Holdout			Average (ACC)
						SBERT	W2V	BOW	SBERT	W2V	BOW	
Majority Class						0.34715			0.37044			0.34093
LogReg	x					0.58034	0.49765	0.54665	0.35870	0.35338	0.36517	0.34093
LogReg	x	x				0.59603	0.53154	0.55602	0.37225	0.37595	0.36097	0.40698
LogReg	x		x			0.63062	0.55748	0.58702	0.40378	0.41290	0.32506	0.42678
LogReg	x			x		0.62972	0.56036	0.58486	0.40352	0.41815	0.33053	0.42886
LogReg	x				x	0.61657	0.55982	0.58846	0.38488	0.38379	0.42198	0.42049
LogReg	x		x		x	0.61818	0.56432	0.59152	0.40967	0.38968	0.37571	0.42940
LogReg	x	x	x		x	0.62484	0.56144	0.59261	0.41534	0.40174	0.36048	0.42441
LogReg	x	x	x	x	x	0.61406	0.55963	0.59009	0.41494	0.39999	0.36104	0.42380
NN	x					0.60249	0.56450	0.60973	0.35736	0.34029	0.36930	0.40922
NN	x	x				0.60540	0.59802	0.61963	0.39011	0.37083	0.35526	0.42006
NN	x		x			0.65800	0.61009	0.65532	0.40633	0.37576	0.34075	0.44411
NN	x			x		0.66070	0.61388	0.66198	0.39309	0.38079	0.33176	0.44500
NN	x				x	0.64017	0.61226	0.62234	0.36721	0.35595	0.40597	0.43395
NN	x		x		x	0.62503	0.61009	0.62198	0.41137	0.39257	0.33095	0.43394
NN	x	x	x		x	0.63206	0.59981	0.62938	0.40885	0.36204	0.39455	0.44276
NN	x	x	x	x	x	0.60557	0.59405	0.61478	0.42270	0.39130	0.35288	0.43002
Average (ACC)						0.62124	0.57468	0.60452	0.39500	0.38157	0.36140	0.33223

Table 1.2
Experimental Results: 95% Monte Carlo Confidence Intervals

	Resp Text	Bundle ID	Quest Text	Context Text	Rubric Text	Monte Carlo CIs		
	x					SBERT	W2V	BOW
NN						(0.3199, 0.4046)	(0.3033, 0.3892)	(0.3323, 0.4157)
NN	x	x				(0.3498, 0.4390)	(0.3078, 0.4774)	(0.3185, 0.4011)
NN	x		x			(0.3371, 0.4905)	(0.2986, 0.4541)	(0.2986, 0.4541)
NN	x			x		(0.3136, 0.4757)	(0.3060, 0.4637)	(0.3060, 0.4638)
NN	x				x	(0.2903, 0.4537)	(0.2798, 0.4375)	(0.2797, 0.4375)
NN	x		x		x	(0.3396, 0.4883)	(0.3194, 0.4786)	(0.3193, 0.4786)
NN	x	x	x		x	(0.3378, 0.4929)	(0.2802, 0.4451)	(0.2802, 0.4450)
NN	x	x	x	x	x	(0.3475, 0.5017)	(0.3079, 0.4774)	(0.3079, 0.4774)

In terms of the general performance of our classification models, we consider the random holdout evaluation method. Overall, it looks as if the SBERT representations performed best when averaging across the classification methods and input combinations, followed by the BOW representations (accuracy of 0.621 for SBERT compared to 0.575 and 0.605 for W2V and BOW, respectively). Additionally, the neural network generally achieves higher accuracy than the logistic regression in general. Both SBERT and BOW seem to perform notably well when the input includes the question text, or the question content.

To assess the generalizability to grading answers to questions unseen in the training set, we focus on the question holdout results. Across the board, we see much lower accuracy for the question holdout experiments than that of the random holdout. This is in line with what one might expect because for the question holdout, the model has not yet seen responses for the particular question in the test set whereas with the random holdout experiments, each question should be represented in the training set (although there is a very low probability that a particular question might not be represented in the training set, due to random chance).

Similar to the random holdout experiments, we see the same overall pattern for the question holdout experiments: SBERT seems generally superior, followed by BOW and W2V, respectively. One notable difference for the question holdout experiments compared to those of the random holdout is that we see increased performance when we include multiple extra sources of information. We might explain this result as, when the model is lacking previous information about the test question from training, extra input information might provide guidance for the model.

We might take inferences from the row averages in the rightmost columns of the table that across all experiments and text representation types, the response and question text, as well as the response and context text achieve the highest evaluation scores. Additionally, the column averages seem to show that the SBERT representations perform best.

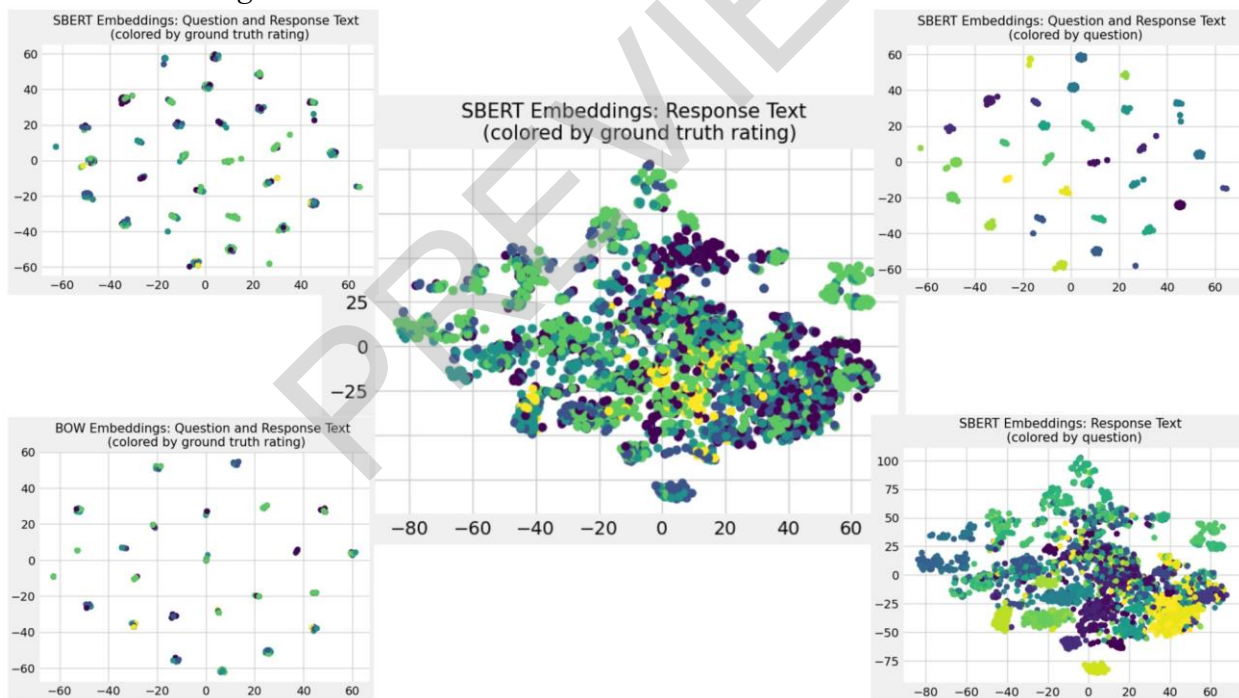
However, the Monte Carlo Confidence Intervals (CIs) in Table 1.2 confirm that no strong conclusions can be made as far as which vectorization method or extra question-related information helps the autograder's performance in terms of accuracy. We see that all, with no exceptions, the empirical confidence intervals overlap.

We include results from a majority class classifier on the top row for a baseline comparison. We emphasize that, across all random holdout experiments, the classification models outperform the majority class classifier significantly. However, this is not the case for the question holdout experiments. For the question holdout experiments, many of the SBERT experiments outperformed the majority class classifier, but when looking at the Monte Carlo confidence intervals, the lower bounds of our condence intervals are generally still below the majority class classification accuracy.

Further interpretations can be made from Figure 1.3, which includes dimension-reduced, t-distributed Stochastic Neighbor Embedding (TSNE) vector representations of student responses. Depending on what text is included as input, we see very different dimension-reduced visualizations of the data. The center image includes SBERT embeddings of only the response text and the colors represent the ground truth ratings. With only response text, it is apparent from the image that the responses do not cluster based on their rating level. However, when comparing the center image to the bottom right image where we color the data points based on the question, we see that even without question text included, the student responses tend to group by question. But, the groupings are not very distinct and overlap throughout the two dimensional space.

Figure 1.3

2D Visuals of Input Vector Representations. Plots Vary by Representation Type, Input Content and Color Labeling.



In contrast, when we include the question text with the response text, the two dimensional vector space looks very different. Referencing the top left, bottom left, and top right images the embeddings are distinctly clustered by question (top right image) but not necessarily by the rating (top left and bottom left image). However, from the bottom right image, we can visualize question specific clusters, but they are not as distinct as the representations that include the question text. From these images, although we are not able to conclude anything about which

vector space is more helpful for an automated grading model, we can see that the embedding space changes dramatically with the inclusion of extra text.

Overall, we cannot conclude that in terms of question holdout, the model can generalize to out-of-sample questions with no clear improvement over the majority class.

1.3.4 Discussion

Although our results are not promising for the generalizability of autograding models to unseen questions, we emphasize the importance of finding more generalizable models to decrease time spent on the laborious task of creating ground-truth human ratings. Our intention is that this work will influence researchers to consider further innovative methods to increase the generalizability of ASAG models. Further, because we did find that including certain question-related text may improve model performance, it may be of use to the ASAG research community to continue to explore how extra sources of information about a question may be incorporated into an ASAG system.

As is evident in our literature review, there has been increased adoption of state-of-the-art textual representation methods such as SBERT, and large language models such as BERT and XLNet, within the field of NLP in Education. Our results support that such models may achieve superior performance for certain tasks.

1.3.4.1 Next Steps

To build on this work further, we could consider other methods, beyond that of concatenation to the input text, to include the extra question information in our model. We could further pre-train a transformer-based model such as BERT or XLNet with the extra textual information by either tuning the existing weights or altering the existing architecture with an extra encoder layer of weights trained on our text alone. Moreover, we may focus more closely on how the classification model itself might be altered such that it might better generalize to out-of-training-sample questions, instead of only focusing on the input content.

We believe that beyond model performance, the practical utility of an ASAG system must be considered in order for educators to continue to adopt new technologies that employ advanced methods in artificial intelligence. Recent years have seen vast improvements in the field of machine learning and language processing. Embracing such technologies for applications in education may be pivotal to provide the assistance that both educators and learners need. However, we do not suggest that machine learning systems such as ASAG should be used to replace human judgements in education, especially in high stakes testing scenarios.

1.4 Chapter 1, Study 2: Utilizing a Scoring Rubric to Increase Model Performance

Many assessments are created via collaboration between teachers, researchers, and test developers where scoring rubrics for each item are carefully designed. Rubrics can provide raters with both exemplar student answers as well as competency descriptions corresponding to each grading category. Rubrics are created to increase both reliability and validity of scores (Waltman et al., 1998). When a rubric clearly and rigidly defines scoring criteria, inter and intra rating consistency can be increased (Wainer, 1993). There are, however, grading scenarios in which rubrics are not created such as contract grading where evaluation is solely based on completion of a task (Danielewicz & Elbow, 2009), or intuitive grading where instructors rely on their experience to grade (Ecclestone, 2001).

In order to exploit the advantages of using scoring rubrics, we propose to incorporate item-specific scoring rubrics into an ASAG model. We do so by further training a model called BERT (Bidirectional Encoder Representations of Transformers) using the scoring rubric text and structure. BERT is a type of language model called a transformer, which has been pre-trained on the English language and can be later trained for downstream tasks such as classification (Devlin et al., 2018). BERT, as-is, encapsulates latent relationships in natural language from the text on which it has been pre-trained. However, we want it to capture such relationships in the scoring rubric text specifically. Before we train it on our classification task of grading, we continue the model’s pre-training with the scoring rubrics. We hypothesize that this will improve the ASAG accuracy beyond that of similar models not trained on rubrics. Additionally, we look at model performance at the question level to examine if certain types of questions and their corresponding rubrics exhibit varied results.

The original BERT model is a neural network consisting of 12 transformer encoder layers with pre-trained weights. To provide the model with the rubric text, we add one more (13th) transformer encoder layer to the model to train with the masked language model (MLM) task, using the rubric text as input. Thus, the model will also incorporate relationships in the text specific to the rubric as it classifies student responses. Additionally, because the ordinality of the rubric is important - namely that its text would not mean much to a human grader if they could not interpret that a level 3 score is ranked “higher” than a level 2 score - we incorporate a sampling method called Node2Vec (Grover & Leskovec, 2016) for creating a vector representation of the rubric text. If we conceptualize the rubric as a graph network to capture the ordinality in the scoring levels, Node2Vec will learn a latent representation of the structure of the rubric.

The novelty of this research is the modification of BERT to incorporate a rubric for each item. The purpose is to increase an automated grading system’s performance for practical use in terms of producing scores that match that of human graders. Using BERT for downstream classification tasks is common for applied machine learning research, as well as for ASAG-specific tasks. Modification of BERT to incorporate a rubric has not yet been studied.

We hypothesize that pre-training BERT on rubric text, termed “further pre-training” will improve the model’s ability to match human ratings of the student responses. However, we do anticipate that this effect may differ depending on the type of questions and their corresponding rubrics.

1.4.1 Background

In this section, we provide a brief background of the BERT model and the Node2Vec sampling method.

1.4.1.1 BERT

BERT (Devlin et al., 2018) was the first language model to successfully learn relationships within sequences of words by pre-training with a bi-directional prediction task, masked language modeling (MLM). It does so by randomly masking 15% of tokens, or words, throughout the text during training and uses a self-attention mechanism to predict the masked texts. Additionally, BERT learns representations between sequences of sentences by its pre-training on a next sentence prediction (NSP) task, where the model is given two sentences and predicts whether one sentence should come directly after the other. The standard BERT model was pre-trained on large amounts of natural language text from Wikipedia and BooksCorpus, and can be fine-tuned for different tasks, such as classification, by adding only one additional layer to

the existing neural network (Devlin et al., 2018). We use a compressed version of the original BERT model called BERT-base, consisting of 12 encoder layers and a total of 110 million parameters. With limited computational resources, the base version is faster to train yet still retains much of the impressive performance on standard benchmark tasks.

1.4.1.2 Node2Vec

Node2Vec is an approach for learning latent representations of vertices in a network such that graph structures can be represented as a continuous vector space (Grover & Leskovec, 2016). By modeling random walks over the graph structure, Node2Vec creates input sequences conceptually similar to that for a sequence of words, however the sequences capture the structure of the graph. The algorithm works as follows: firstly, a random walk generator takes in a graph structure, and uniformly samples a node as the root of the walk. Then, the walk uniformly samples from the direct neighbors of the last visited node, and this process continues until a maximum length has been reached. This sequence of nodes would represent one input sequence, and as many sequences can be generated as specified. In addition to the number of walks and the length of walks, two other parameters exist which specify how likely it is for the walk to wander far from the starting node (breadth of the search), and how likely it is to return to the start node (depth of the search).

1.4.2 Methods

In this section, we describe the data used for this project, outline the methods used to alter the architecture of the BERT-base model, explain the Node2Vec sampling approach for creating input sequences to further pre-train the extra layer of the model, and detail the process of training the classifier.

1.4.2.1 The Dataset

We used two data sets for this project, consisting of short responses to science questions. The first dataset we used was sourced from a 2019 field test of an assessment created at the Berkeley Evaluation and Assessment Research (BEAR) center. Further details of this dataset can be found in section 1.3.1.

We also show results with an open-source data set called the Automatic Student Assessment Prize (ASAP) Short Answer Scoring (SAS) data (Hamner et al., 2012). The data was used in a 2012 Kaggle competition sponsored by the Hewlett Foundation, and consists of almost 13,000 short answer responses to 10 science and English questions. We used only the 5 science questions since the domain is the same as the BEAR data set. The questions were scored from 0 (incorrect) to 3 (fully correct), and each question includes a scoring rubric. We refer to this data set as ASAP. An example of a question from the ASAP data set is:

"Starting with mRNA leaving the nucleus, list and describe four major steps involved in protein synthesis."

The rubric consists of a list of key elements of an exemplary response such as:

"mRNA exits the nucleus via nuclear pore"

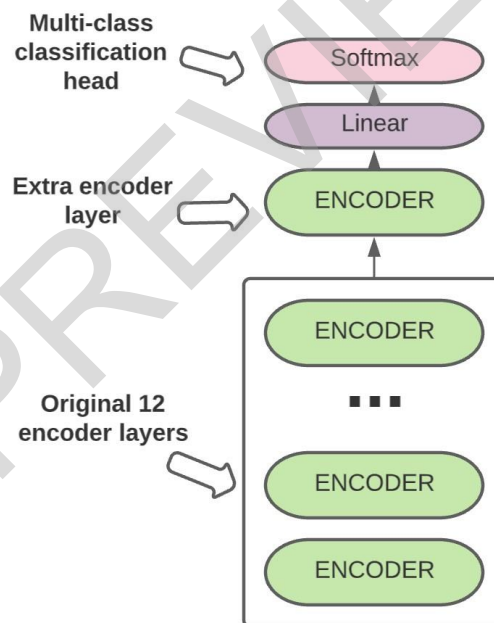
and specifies how many key elements need to be included for the answer to be rated at a given level.

Standard pre-processing of both the data sets included removing null responses, adding a period to the end of sentences, and concatenating a numeric question identifier to the beginning of student responses. We split the data sets into training, validation and testing sets with a 70/15/15 percent split.

1.4.2.2. Architecture Modification

To further pre-train the model on the scoring rubric text, we add a 13th encoder transformer layer to the original BERT-base model. The weights of the 13th layer are randomly initialized, and the weights of the other 12 encoder layers are frozen during the additional pre-training. Thus, the original 12 layers maintain the weights that capture latent relationships in general English language from the model's original pre-training, and the new weights of the 13th layer start as essentially a blank slate. We then train the 13th layer with the MLM task using our scoring rubric text as input. Standard neural network parameters were tuned to minimize the MLM loss, in addition to considering the best configuration for our computational limitations. Optimal parameters include a batch size of 8, a learning rate of $4e-4$, and 12 training epochs. Figure 1.4 includes an illustration of the architecture of the modified BERT model.

Figure 1.4
The Modified BERT Architecture



1.4.2.3 Sampling Methods

To produce input sequences that capture the ordinal structure of the scoring rubric for pre-training the BERT model, we use the Node2Vec sampling method. We create a graph representation of the structure of the scoring rubric for each question in the data set. An example of a scoring rubric and its corresponding graph structure is included in Figure 1.5. In order to conceptualize the rubric as a graph, we make each level of the scoring rubric a node in the graph, and undirected edges connect each level to both the level below and above it. The description of